

Karachi Air Quality Index Prediction System

An End-to-End Serverless Machine Learning Pipeline

Final Project Report

Submitted by

Sameen Zaki

10Pearls Shine Internship Program — Cohort 9

Data Sciences

September 2026

Contents

List of Figures	iv
List of Tables	vi
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	1
2 Project Scope	3
2.1 In Scope	3
2.2 Out of Scope	3
3 Tools and Technologies	4
4 System Architecture	5
5 Data Collection	7
5.1 Sources	7
5.2 Historical Backfill	7
5.3 Live Collection	7
6 Data Cleaning and Preprocessing	8
7 Exploratory Data Analysis	9
7.1 Simple Plot	9
7.2 AQI Category Understanding	10
7.3 Correlation Understanding	11
7.4 Yearly Minimum and Maximum	12
7.5 Temporal Patterns	13
7.6 Seasonal Decomposition	14

7.7	Auto Correlation Function	16
7.8	Missing Values	16
8	Feature Engineering	18
8.1	Feature Categories	19
8.2	Preventing Look-Ahead Bias	20
8.3	Target Construction	20
9	Feature Store	22
10	Model Development	23
10.1	Validation Strategy	23
10.2	Baseline	23
10.3	Day-1 Model	23
10.4	Day-2 and Day-3 Models	23
10.5	Alternative Approaches Evaluated	24
11	Results and Evaluation	25
11.1	Deployed Model Performance	25
11.2	Full Model Comparison	25
12	Registry, Automation and Deployment	26
12.1	Model Registry and Promotion Guard	26
12.2	Automation	26
12.3	REST API	26
12.4	Web Application	27
12.5	Application Screenshots	28
12.6	Interpretability and Alerting	31
13	Problems Encountered and Solutions	34
13.1	Data Collection and Cleaning	34
13.2	Feature Engineering and Leakage	34
13.3	Model Performance	35
13.4	Feature Store and Infrastructure	35
13.5	Deployment	36

13.6	Application Layer	37
14	Conclusion	38
15	References	39

List of Figures

1	End-to-end system architecture. Solid arrows show the primary data flow; dashed arrows show read access from the serving layer.	5
2	Project lifecycle across ten stages, from problem definition through to serving and monitoring.	5
3	Daily mean US AQI for Karachi, August 2023 to July 2026.	9
4	The same series zoomed to April–September 2025.	9
5	Count of days falling into each US EPA AQI category.	10
6	Correlation matrix across all engineered features.	11
7	Maximum AQI recorded in each year.	12
8	Minimum AQI recorded each year.	13
9	Mean AQI by day of the week.	13
10	Mean AQI by month.	14
11	Seasonal component from additive decomposition (period = 365).	14
12	Trend component from additive decomposition.	15
13	Residual component from additive decomposition.	15
14	Autocorrelation function of daily AQI.	16
15	Null value count for every column in the daily dataset.	16
16	Autocorrelation of daily AQI. Significant correlation persists beyond thirty days.	20
17	Home page: current AQI with health category, and current temperature.	28
18	Home page: one-, two- and three-day forecast cards above the recent trend chart.	28
19	Dashboard: US EPA AQI category legend with colour coding.	29
20	Dashboard: historical AQI trend with a user-selectable window.	29

21	Dashboard: selectable EDA plots, here showing the full three-year AQI series.	29
22	Model Insights: registry metrics for the Day-1 and Day-2 models, including baseline comparison.	30
23	Model Insights: Day-3 metrics. The negative baseline R^2 of -0.08 and the resulting gain of 0.59 are both surfaced.	30
24	Model Insights: SHAP analysis selector showing a Day-1 waterfall decomposition.	30
25	Analyze page: batch backtesting upload interface.	31
26	SHAP beeswarm for the Day-1 model. pm2_5 is the dominant driver. . .	31
27	SHAP beeswarm for the Day-2 model.	32
28	SHAP beeswarm for the Day-3 model.	32
29	SHAP waterfall for an individual Day-1 prediction.	33

List of Tables

1	Tools and technologies used in the project	4
2	The four stages of the system	6
3	Variables collected from the two Open-Meteo endpoints	7
4	Domain bounds applied during data cleaning	8
5	Engineered feature categories	19
6	Datasets produced by the feature pipeline	21
7	Hopsworks feature groups and feature views	22
8	Day-1 XGBoost hyperparameters	23
9	Components of the Day-2 and Day-3 blends	24
10	Registered model performance against the persistence baseline	25
11	Comparison of all models at hourly and daily granularity	25
12	REST API endpoints	27
13	Pages of the Streamlit dashboard	27

1 Introduction

1.1 Background

Air pollution is one of the most common health problems everyone faces, especially in urban areas. Karachi, where most of the population of Pakistan lives must experience air quality that almost ranges into unhealthy categories due to vehicles gas emissions, industrial activity, construction and dust particles trapping pollutants closer to the ground level.

The Air Quality Index is a scale that helps you to understand the atmosphere changes without having any technical knowledge. This project adopts the US EPA AQI standard, which maps all the pollutants onto 0 to 500 scale dividing it into 6 health categories. It was selected because it is recognized globally and is available in Open-Mateo.

1.2 Problem Statement

The problems that are solved in this project are automation, continuous updating, and multiple day forecasting system for Karachi. The efforts require more than just training a model on a dataset. AQI Forecasting System must ingest new coming observations continuously, and retrain too as per observations updates, also maintain the gateway in training pipeline so that a degraded model can be rejected, and serve predictions easily on a web interface so that non-technical users can understand.

1.3 Objectives

- Collect 3 years of historical data of weather and AQI features for Karachi and establish the data pipeline which runs after every one hour and accumulates the new observations.
- Now, feature engineering includes temporal cycles, rolling statistics, recent history and domain specific features.
- After feature engineering and cleaning process, the data is stored in a feature store, so that the training models can be read from an identical source.
- Training the models for day one, two, and three. Evaluating them against metrics especially persistence baseline.
- The best trained models are registered in Hopsworks model registry along with their evaluation metrics. And guarding the model registry so the worst model won't be registered.

- Automating feature collection and model training on a schedule using GitHub Actions (CI/CD).
- Finally, deploying Fast Api and a streamlit dashboard presenting current AQI and temperature, forecasts, health alert, eda, model analysis, and interpretability analysis.

2 Project Scope

2.1 In Scope

- Model interpretability through SHAP analysis, and health alerting for hazardous AQI levels.
- Geographic coverage: Karachi at latitude 24.8608, longitude 67.0104.
- On a daily average, forecast horizons day one, two and three.
- Individual pollutant concentrations are used as an x (input) features and US EPA AQI as the single prediction target.
- Automating feature collection and model retraining on a schedule.
- A public Fast Rest Api and an interactive dashboard, both deployed on free tier.

2.2 Out of Scope

- The feature engineering is specifically designed for Karachi city predictions not for multiple cities.
- Hourly based models were built and evaluated too but the deployed models work on daily granularity, for reasons documented in Section 8.
- Models cannot forecast up to 3 days as the predictions can degrade this point on the available data.

3 Tools and Technologies

Tools and technologies are used as follows:

Table 1: Tools and technologies used in the project

Category	Technology
Language Used	Python 3.11
Weather Data source	Open-Meteo Forecast API
AQI Data source	Open-Meteo Air Quality API
Data handling	pandas 2.3.3, NumPy 2.3.5
Feature store	Hopsworks 5.0.6 (free tier)
Model registry	Hopsworks Model Registry
XGBoost Model	XGBoost 3.4.0
CatBoost Model	CatBoost 1.2.10
Other Models	scikit-learn 1.7.2
LSTM Model	TensorFlow / Keras
Prophet Model	Prophet
Interpretability	SHAP
Automation (CI/CD)	GitHub Actions
Backend	FastAPI + Uvicorn
Frontend	Streamlit
Hosting	Render (free tier)
Monitoring	Uptime Robot
Configuration	python-dotenv
Version control	Git / GitHub

4 System Architecture

The system architecture is organized into four stages, and every stage is linking or you can say communicating through the feature store and the model registry. This means each stage can be run, tested and debugged separately and failure in one stage does not affect others.

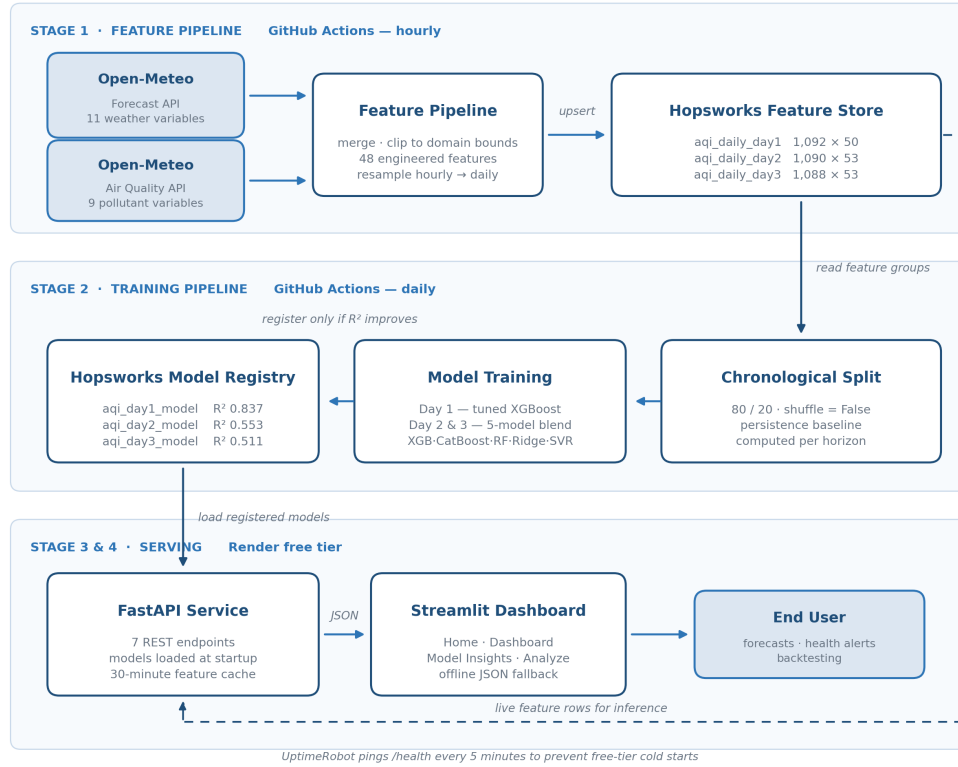


Figure 1: End-to-end system architecture. Solid arrows show the primary data flow; dashed arrows show read access from the serving layer.

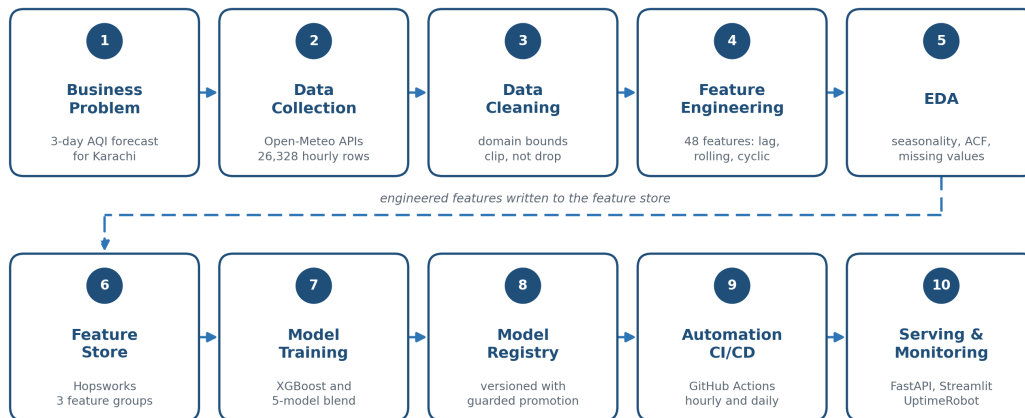


Figure 2: Project lifecycle across ten stages, from problem definition through to serving and monitoring.

Table 2: The four stages of the system

Stage	Component	Trigger	Output
1. Feature Pipeline	functions.py, data_pipeline.py	GitHub Actions runs hourly	Updated rows go into three feature groups
2. Training Pipeline	train_functions.py, train_models.py	GitHub Actions runs daily	Three models in model registry of Hopsworks
3. Rest API	api_functions.py, main.py (FastAPI)	HTTP request	Predictions, metrics, history and predict file results
4. Web Application	app.py (Streamlit)	User interaction	Dashboard with horizons forecasts, current results, EDA, model insights, and predict file results

The flow of data throughout the system is in one direction. The feature pipeline writes to feature store. The training pipeline reads the data from the feature groups and updates the model registry. The API reads but writes to none of them, this is why the training pipeline needs to be rerun at any time without risk to the serving path, allowing API to be redeployed without any data modification.

5 Data Collection

5.1 Sources

Two open sources of Open-Meteo endpoints give all the data, it was selected over other API alternatives because it requires no API key, provide both the historical and forecasting data through a single interface, returning a precomputed US EPA AQI value.

The weather API gives eleven hourly based features. And the air quality API supplies hourly based pollutants variables. Then both weather and air quality data are merged on the timestamp column.

Table 3: Variables collected from the two Open-Meteo endpoints

Group	Variables
Meteorological (11)	temperature_2m, wind_speed_10m, relative_humidity_2m, surface_pressure, boundary_layer_height, dew_point_2m, precipitation, cloud_cover, wind_direction_10m, wind_gusts_10m, shortwave_radiation
Pollutants (8)	pm10, pm2_5, carbon_monoxide, nitrogen_dioxide, sulphur_dioxide, ozone, dust, aerosol_optical_depth
Target (1)	us_aqi

5.2 Historical Backfill

Historical collection is covering from August 1 of 2023 to August 1 of 2026. Three years were chosen so the dataset would have three complete winter smog seasons, giving model more than a single example of each seasonal transition.

5.3 Live Collection

In production, the system uses the rolling window instead of relying on the historical data itself. It gets the data of past 92 days and 7 days ahead. The future weather data is also essential because the model needs **temperature, wind speed, and humidity** to predict AQI. So if we only rely on past data the future weather values will be missing and that would make impossible to generate live AQI predictions.

6 Data Cleaning and Preprocessing

So, in this project, we did not use any statistical method for outlier handling because it could reduce the model’s ability to learn the severe pollution patterns. I first evaluated it on IQR based outlier handling but then rejected it because extreme values are often the real pollution events.

So instead, I used real world-based limits based on Karachi’s atmospheric behavior, and values outside these ranges are clipped.

Table 4: Domain bounds applied during data cleaning

Variable	Lower bound	Upper bound	Justification
temperature_2m	0 °C	55 °C	Karachi’s most of the recorded values lie in this range
relative_humidity_2m	0 %	100 %	The definition of relative humidity
surface_pressure	870 hPa	1085 hPa	The observed range of sea level pressure
dew_point_2m	-20 °C	40 °C	Bounded by regional climate
cloud_cover	0 %	100 %	Percentage by definition
wind_direction_10m	0°	360°	Compass bearing by definition
All pollutants, us_aqi	0	unbounded	Negative concentrations are sensor error; high values are real events

We used clipping instead deleting the rows because time series depends on continuous sequence of the hourly data. As the model depends on **lag features, rolling statistics, and shifted target values**. So, the more rows deletion can affect the calculations and give wrong feature values. Clipping allows us to keep all the features maintaining the sequence of time series.

7 Exploratory Data Analysis

Exploratory Data Analysis was carried out as follows:

7.1 Simple Plot

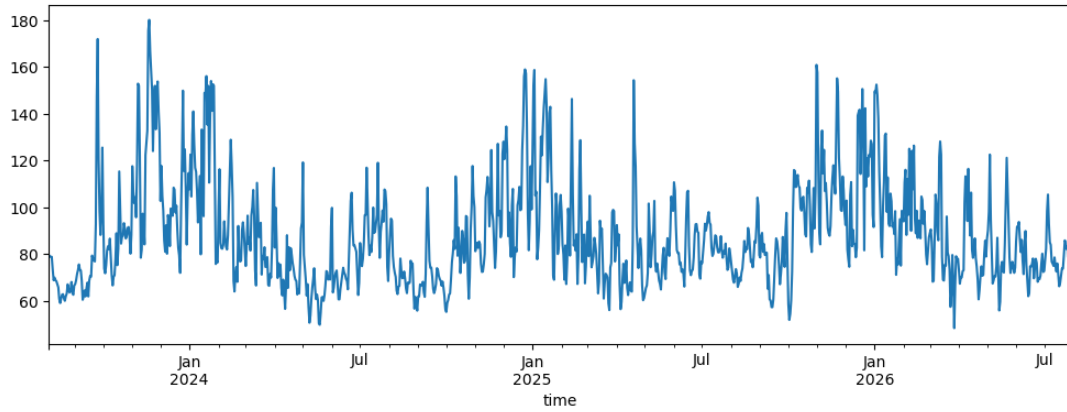


Figure 3: Daily mean US AQI for Karachi, August 2023 to July 2026.

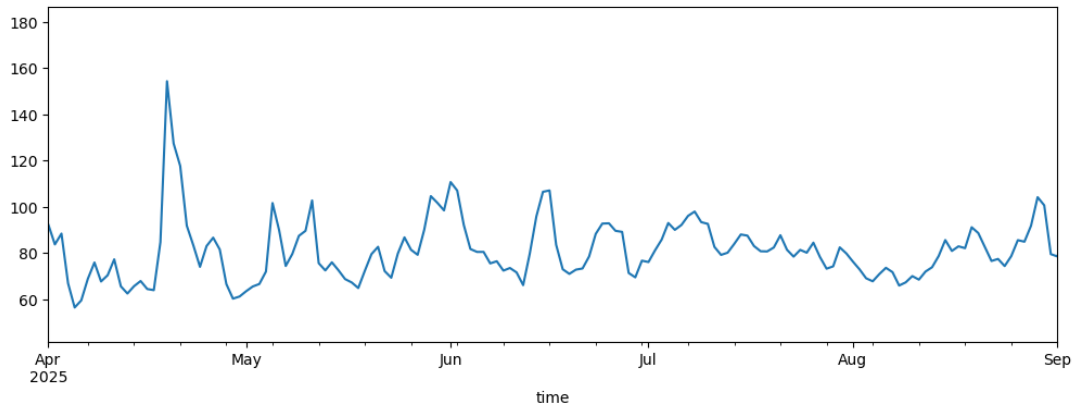


Figure 4: The same series zoomed to April–September 2025.

Note:

1. The AQI is low from March to September, and November to February high.
2. There is a seasonal pattern.
3. The highest AQI is 180 and lowest is 48.

7.2 AQI Category Understanding

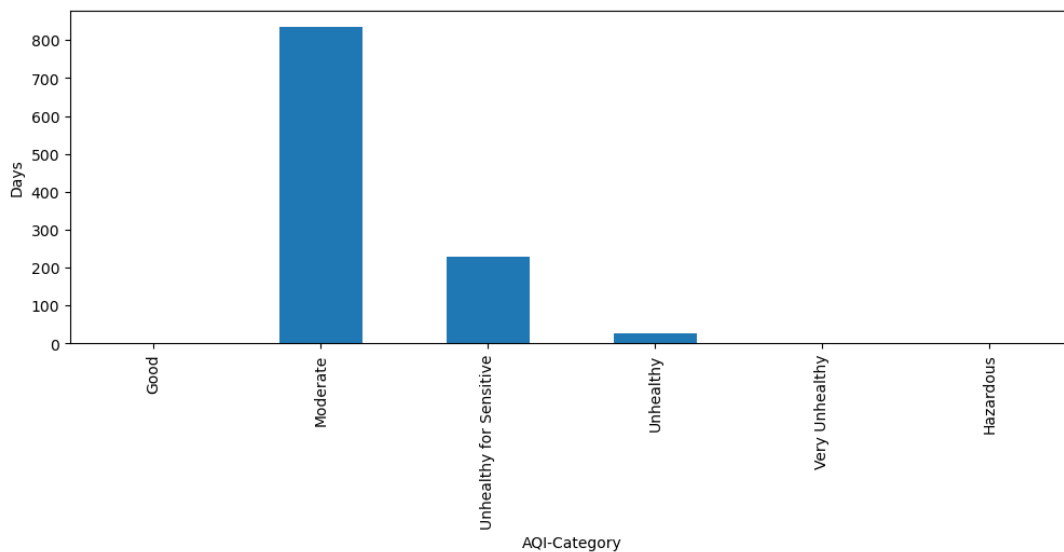


Figure 5: Count of days falling into each US EPA AQI category.

Note:

1. Moderate (51-100): 76.5% of hours have a moderate AQI level. This is Karachi's typical air quality.
2. Unhealthy for Sensitive (101-150): Unhealthy for Sensitive is about 20.88%, meaning sensitive groups face air quality concerns roughly one-fifth of the time.
3. Unhealthy (151-200): Unhealthy is about 2.47%, these are short pollution events.
4. Good (0-50): Good is about 2 days (0.18%) so the clean air is extremely rare.
5. No record for Very Unhealthy and Hazardous.

7.3 Correlation Understanding

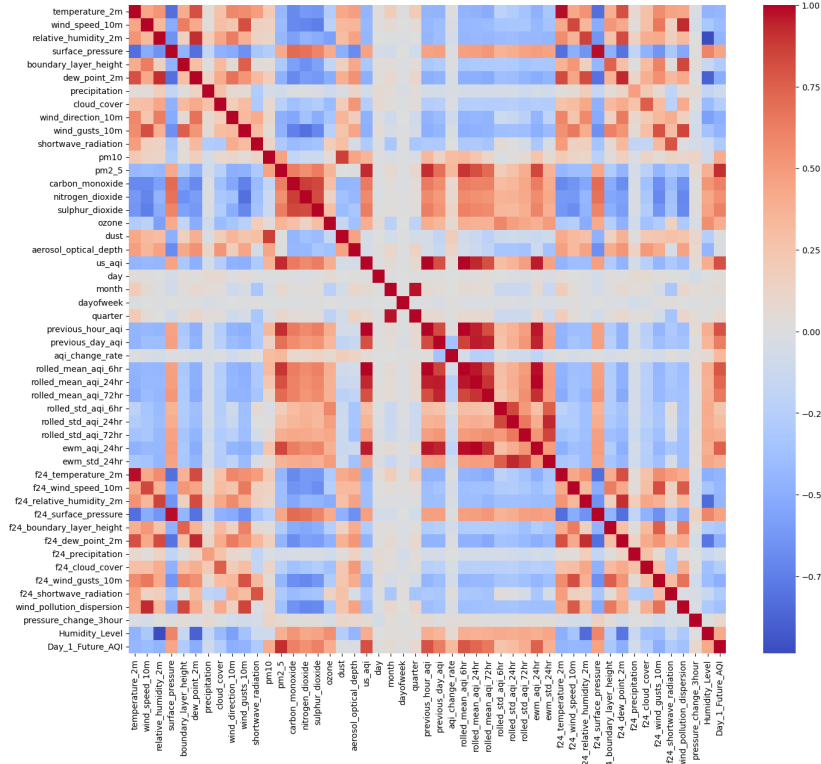


Figure 6: Correlation matrix across all engineered features.

Note:

1. The strongest predictors of the target are `previous_day_aqi` and the rolling mean features (`rolled_mean_aqi_6hr`, `24hr`, `72hr` and `ewm_aqi_24hr`). This validates the features of engineering choices.
2. This lag and rolling features are also highly correlated with each other, since all are smooth versions of the same AQI series. This multicollinearity does not break tree-based models, but it does split feature importance across near-duplicate columns, and it would be a problem for linear regression.
3. Pollutant correlations are structural rather than informative. `us_aqi` is calculated from `pm2_5`, `pm10`, `ozone` and the other pollutants, so their high correlation only reproduces the AQI formula and reveals nothing new about the data.
4. Calendar features (`day`, `dayofweek`, `month`, `quarter`) show near-zero linear correlation with AQI. This does not mean they are useless. Pearson correlation measures straight-line relationships and cannot capture a cycle, where high values return periodically. The month bar chart proves the seasonal effect is strong (January 113 vs August 73) even though the correlation is close to zero.

5. Wind speed correlates negatively with AQI, since stronger wind disperses pollutants.
6. `previous_hour_aqi` still appears in this daily dataset. It was carried over from the hourly pipeline and has no meaning at daily resolution, so it should be dropped or recomputed.
7. The heatmap was originally plotted with `cmap="Paired"`, a qualitative colormap whose colours have no order, which made the values impossible to compare visually. It has been changed to a diverging colormap centred at zero.

7.4 Yearly Minimum and Maximum

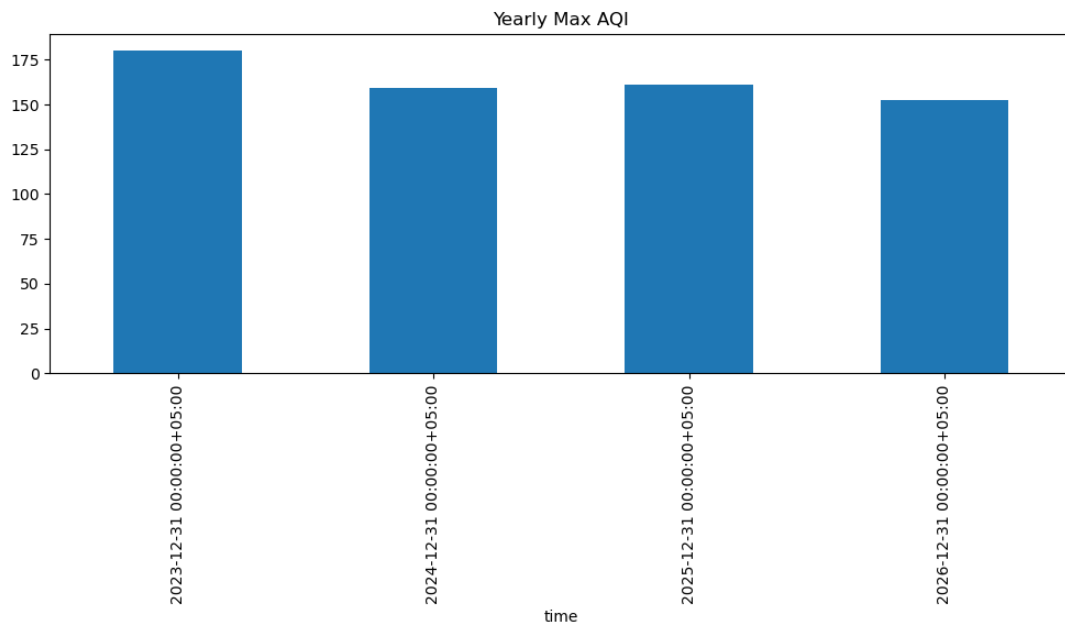


Figure 7: Maximum AQI recorded in each year.

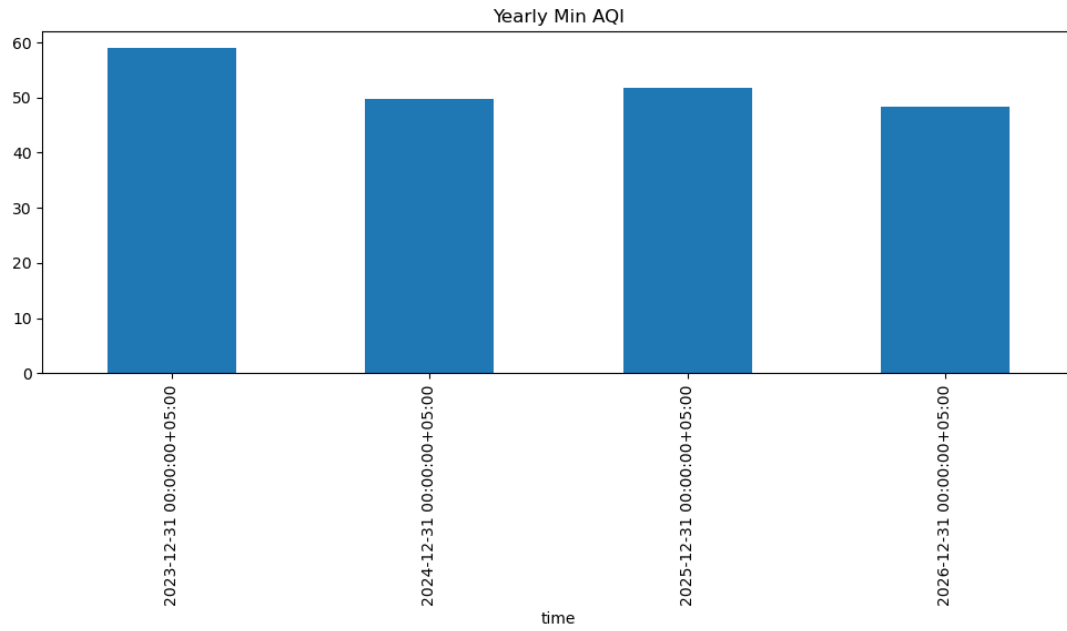


Figure 8: Minimum AQI recorded each year.

Note:

- Both charts appear to show declining trend.
- Yearly max AQI are 180 - 2023, 159 - 2024, 161 - 2025, 153 - 2026 as numbers guessed from chart.
- Yearly min AQI are 59 - 2023, 50 - 2024, 52 - 2025, 48 - 2026 as numbers guessed from chart.

7.5 Temporal Patterns

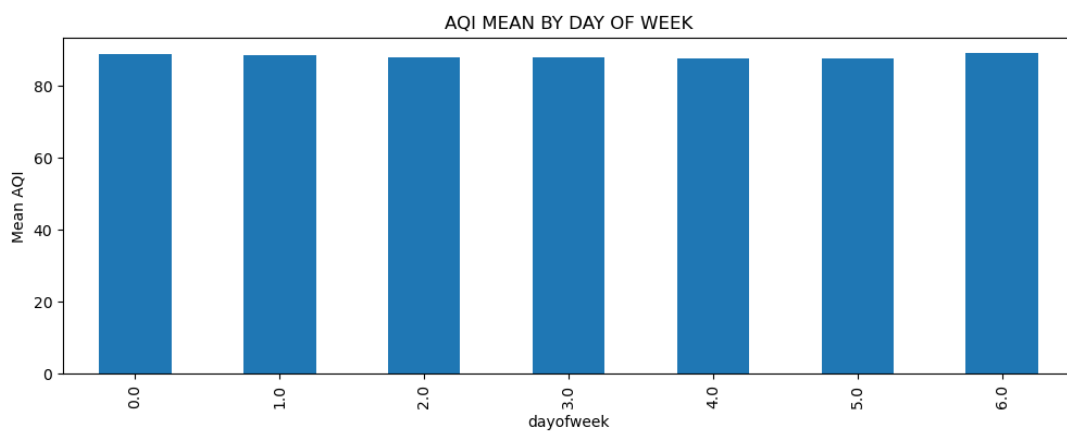


Figure 9: Mean AQI by day of the week.

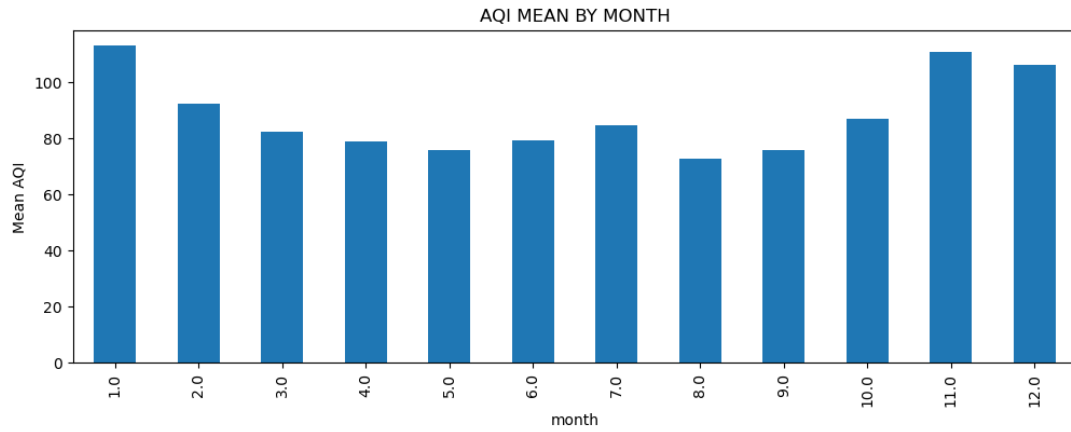


Figure 10: Mean AQI by month.

Note:

1. All 7 days in a week the average is between 87 to 89 AQI, which shows no effect.
2. Month seems to be the having strongest impact as January is the highest and August is the lowest.

7.6 Seasonal Decomposition

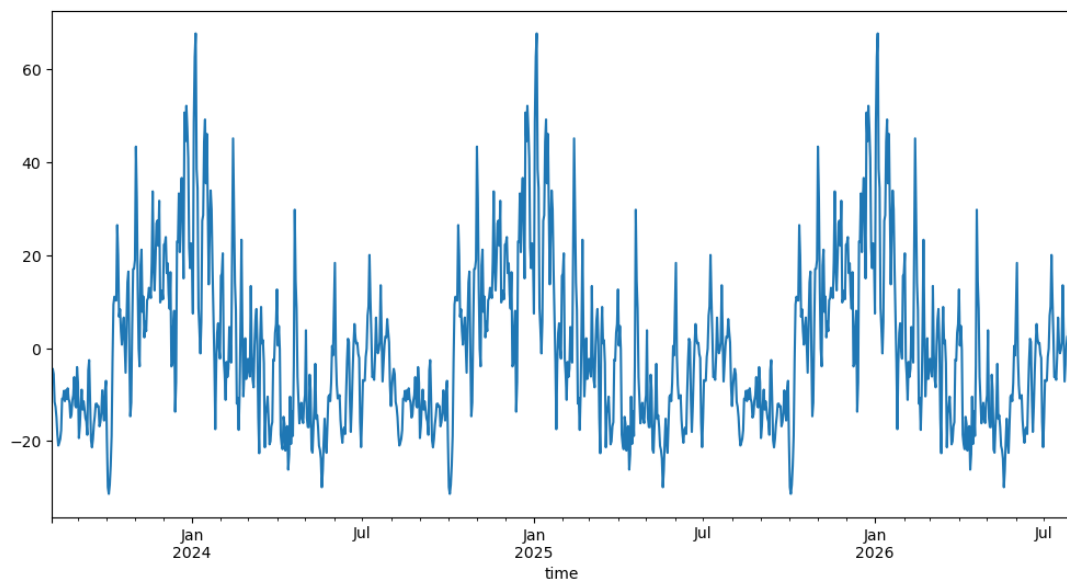


Figure 11: Seasonal component from additive decomposition (period = 365).

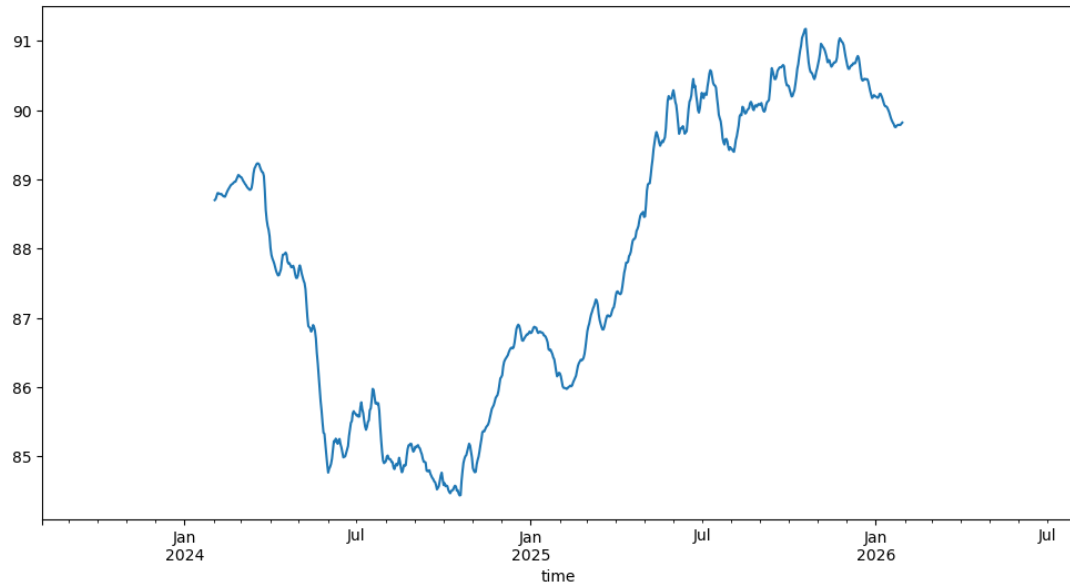


Figure 12: Trend component from additive decomposition.

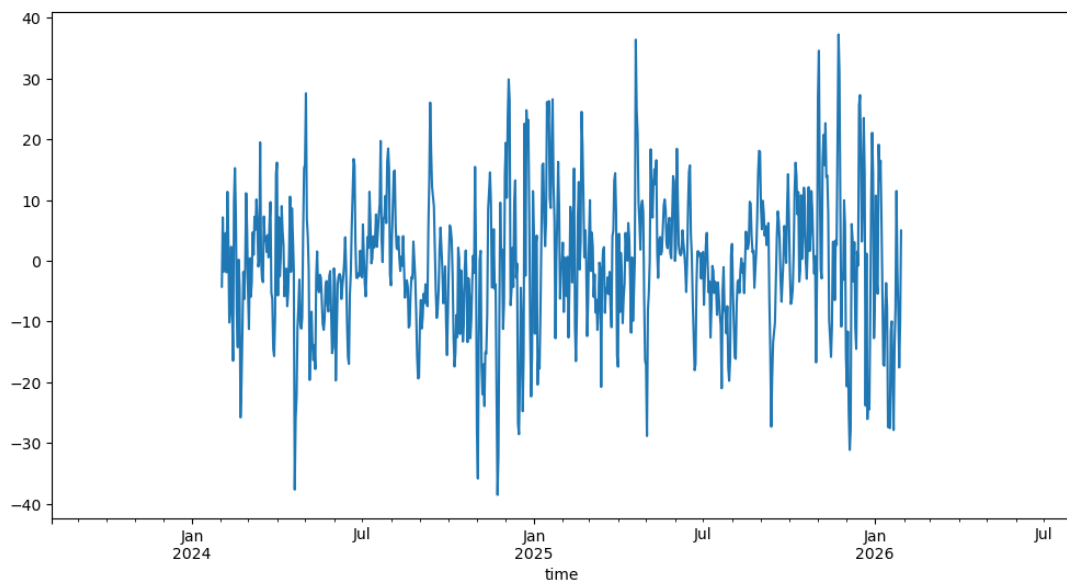


Figure 13: Residual component from additive decomposition.

Note:

The annual cycle is now in the seasonal component, and the trend is almost flat, so AQI has not meaningfully changed over three years. The residual is large and more volatile in winter.

7.7 Auto Correlation Function

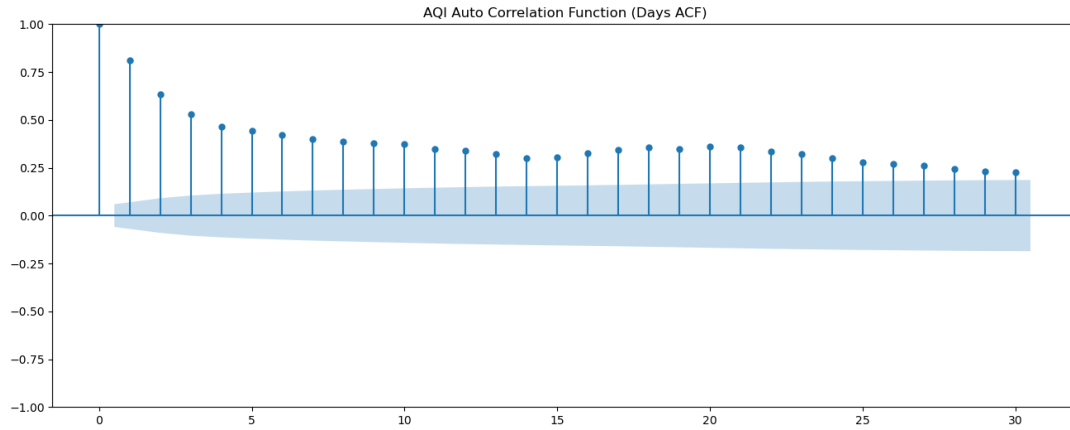


Figure 14: Autocorrelation function of daily AQI.

Note:

1. Lag 24 has a correlation almost to 0.81.
2. Lag 48 has a correlation almost to 0.63.
3. Lag 72 has a correlation almost to 0.53.

7.8 Missing Values

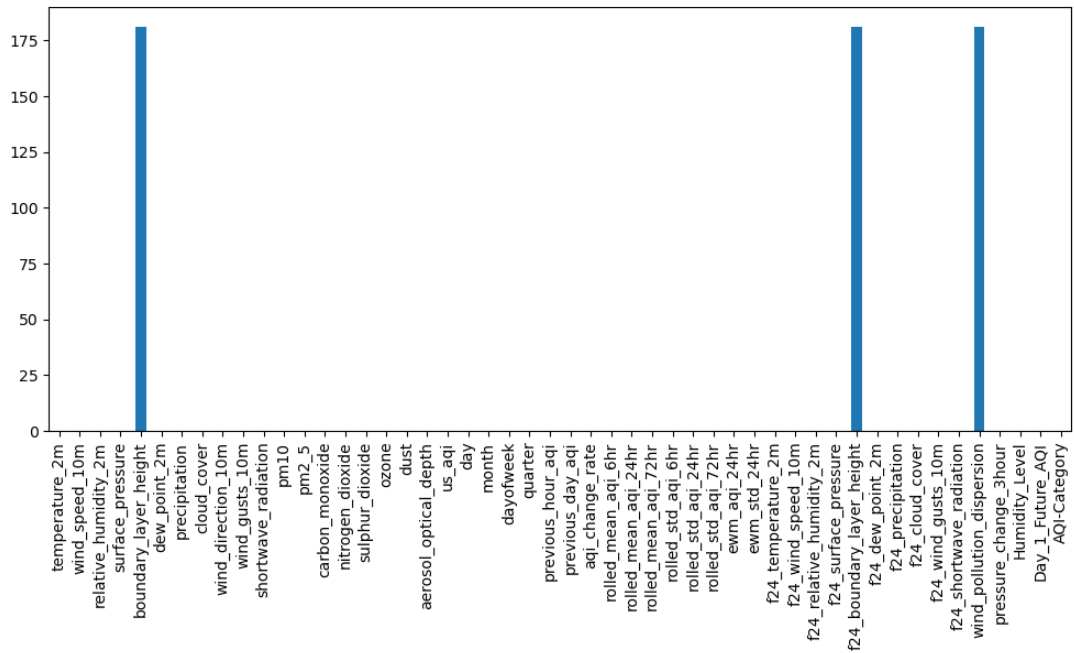


Figure 15: Null value count for every column in the daily dataset.

Note:

boundary_layer_height, f24_boundary_layer_height, and wind_pollution_dispersion have above 174 null values.

8 Feature Engineering

Features are built based on hourly and then aggregated to daily. Initially building it hourly was important because lag and rolling variables are computed on hourly data that's why.

8.1 Feature Categories

Table 5: Engineered feature categories

Category	Features	Rationale
Temporal	hour, day, month, dayofweek, quarter	It helps the model understand the seasonal patterns of daily, monthly and weekly.
Lag	previous_hour_aqi (shift 1), previous_day_aqi (shift 24)	Lag uses recent values of AQI because it is strongly related to past AQI.
Derived	aqi_change_rate (first difference)	It captures whether the AQI is getting worse or improve.
Rolling statistics	rolling mean and standard deviation over 6-, 24-and 72-hour windows	It helps in understanding short term smoothing and showing how much the AQI has changed.
Exponential	ewm_aqi_24hr, ewm_std_24hr (span 24)	Gives more weight to current values.
Future weather	f24_, f48_, f72_ prefixed versions of ten weather variables	Gives the model future values of the weather.
Domain	wind_pollution_dispersion (wind speed $\times$ boundary layer height), pressure_change_3hour, humidity_level (temperature - dew point)	Based on real physical factors on how pollution can spread and accumulate.
Cyclic (Day 2/3 only)	cyclic_day_sin/cos (annual), cyclic_week_sin/cos (weekly)	Continuous encoding so that December and January are numerically adjacent
Delta weather (Day 2/3 only)	Difference between forecast and current values for five weather variables	Rather than just showing the AQI, it shows the expected change in AQI.

The ACF chart shows the justification on how the past AQI values are useful for forecasting future AQI. Correlation of AQI is positive for 30 lags and more than that, so this correlation information is significantly useful.

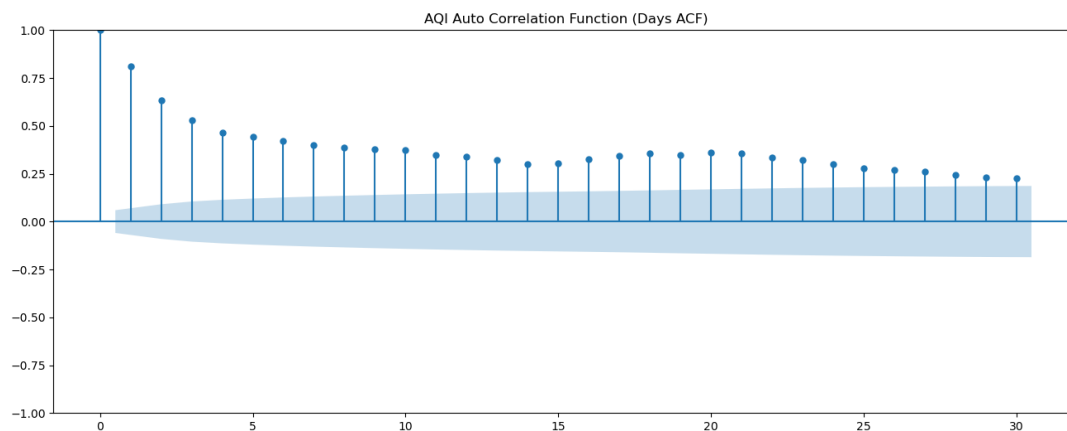


Figure 16: Autocorrelation of daily AQI. Significant correlation persists beyond thirty days.

8.2 Preventing Look-Ahead Bias

The rolling and exponential variables are calculated based on shifting by one period of hour because the rolling and exponential at time t included the observation at t too. The reason is the AQI at time t is too close to the predicted target, the model will be overfitted giving great validation scores and useless forecasts.

8.3 Target Construction

On hourly basis three targets were generated which were shifted by shifting the target to 24, 48 and 72 rows ahead. After doing the daily resample the targets are finalized to day one, two and three. So, each horizon has its own dataset where the other two are removed.

Table 6: Datasets produced by the feature pipeline

Dataset	Rows	Columns	Date range	Target
Hourly, raw merged	26,328	21	2023-08-01 to 2026-08-01	—
Hourly, Day-1 engineered	26,232	51	2023-08-04 to 2026-07-31	Day_1_Future_AQI
Daily, Day-1	1,092	50	2023-08-04 to 2026-07-30	day_1_future_aqi
Daily, Day-2 (cyclic)	1,090	53	2023-08-04 to 2026-07-28	day_2_future_aqi
Daily, Day-3 (cyclic)	1,088	53	2023-08-04 to 2026-07-26	day_3_future_aqi

The counts of rows differ because the first 72 rows are lost to the longest rolling window, and the final 24, 48 or 72 rows are lost because their future targets do not yet exist.

9 Feature Store

Hopsworks was selected as the feature store because free tier has both the model registry and feature store functionality for one project only, without worrying about the need of two services. The project is hosted under a unique name **aqi_karachi_samu_2026**. Three feature groups are created and maintained for each horizon with the time columns as both primary key and event. Using it as a primary key means the data pipeline can do upsert rather than append (which means updating the rows instead of duplicating it).

Table 7: Hopsworks feature groups and feature views

Feature group	Rows	Columns	Feature view	Label
aqi_daily_day1	1,092	50	view_aqi_day1	day_1_future_aqi
aqi_daily_day2	1,090	53	view_aqi_day2	day_2_future_aqi
aqi_daily_day3	1,088	53	view_aqi_day3	day_3_future_aqi

10 Model Development

10.1 Validation Strategy

The models used `train_test_split` on eighty twenty split techniques with `shuffle` set to `false`. Eighty percent is the training data and the rest is testing. If random shuffling happened then the future data would come in the training data and past data would come in the testing because of this model would achieve high score, but in real world it does not happen like that. The chronological split is used in the models due to the above stated reason.

10.2 Baseline

During the model evaluation it is compared against its persistence baseline which is the naïve forecast predicts the AQI based on the current AQI. Without this the R2 score can be misleading as comparing it with the baseline shows how much the model has actually learned.

10.3 Day-1 Model

For Day One Model XGBoost regressor is used, and the best parameters were selected through `RandomizedSearchCV` with `TimeSeriesSplit` cross-validation tuning.

Table 8: Day-1 XGBoost hyperparameters

Hyperparameter	Value	Hyperparameter	Value
<code>n_estimators</code>	1800	<code>subsample</code>	0.8
<code>learning_rate</code>	0.01	<code>colsample_bytree</code>	1.0
<code>max_depth</code>	6	<code>reg_alpha</code>	0.1
<code>min_child_weight</code>	10	<code>reg_lambda</code>	1.0
<code>gamma</code>	0.4	<code>tree_method</code>	hist

10.4 Day-2 and Day-3 Models

Single model was not performing best so blend technique was used of five learners.

Table 9: Components of the Day-2 and Day-3 blends

Component	Configuration	Input
XGBoost	520 estimators, lr 0.033, depth 3, subsample 0.7, colsample 0.7	Raw features
CatBoost	3000 iterations, lr 0.02, depth 4, L2 leaf reg 10, early stopping 100	Raw features
RandomForest	500 trees, depth 8, min_samples_leaf 10, max_features 0.5	Raw features
Ridge	alpha 1.0	StandardScaler-transformed
BaggingRegressor(SVR)	500 estimators, RBF kernel, C 100, epsilon 1.0, max_samples 0.25	StandardScaler-transformed

10.5 Alternative Approaches Evaluated

Three alternatives' models were also built and rejected as justified below:

LSTM. A two layered LSTM model was built **64 and 32 units**, 0.3 dropout, and a 24-hour lookback was tested on the hourly Day-1 dataset. It crossed the R2 score 0.59 against the hourly persistence baseline of 0.378 but failed to beat XGBoost at 0.625. Also, it has overfitting as the training loss was lower than the validation loss. Also, LSTM requires huge amounts of data so I can not pass the daily aggregated data instead I gave hourly ones around 26,000 rows still not sufficient.

Prophet. Facebook Prophet with the regressors achieved R2 score 0.517 but the simple one just crossed 0.003. As our data do not have strong trends, so the Prophet was not suitable for this data.

CatBoost as the Day-1 model. CatBoost achieved R^2 0.840 which is little more than the XGBoost R2 score. However, the difference is insignificant, XGBoost is selected because of smaller model file and lower memory usage.

11 Results and Evaluation

11.1 Deployed Model Performance

Currently deployed model metrics are as follows:

Table 10: Registered model performance against the persistence baseline

Horizon	Model	R ²	MAE	RMSE	Baseline R ²	Gain
Day 1	Tuned XGBoost	0.837	5.37	7.64	0.552	+0.285
Day 2	5-model blend	0.553	9.16	12.67	0.190	+0.363
Day 3	5-model blend	0.511	9.84	13.29	-0.081	+0.592

11.2 Full Model Comparison

All model evaluations are as follows:

Table 11: Comparison of all models at hourly and daily granularity

Model	R ²	MAE	MSE	RMSE
XGBoost Day 1 (Hourly)	0.625	8.538	150.763	12.278
XGBoost Day 2 (Hourly)	0.349	11.754	261.561	16.172
XGBoost Day 3 (Hourly)	0.292	12.259	284.177	16.857
XGBoost Day 1 (Daily)	0.836	5.369	58.324	7.637
XGBoost Day 2 (Daily)	0.555	9.108	159.790	12.640
XGBoost Day 3 (Daily)	0.503	9.841	179.462	13.396
CatBoost Day 1 (Hourly)	0.587	9.087	165.932	12.881
CatBoost Day 2 (Hourly)	0.381	11.386	248.690	15.769
CatBoost Day 3 (Hourly)	0.331	11.869	268.838	16.396
CatBoost Day 1 (Daily)	0.840	5.383	57.090	7.555
CatBoost Day 2 (Daily)	0.540	9.320	163.374	12.780
CatBoost Day 3 (Daily)	0.504	9.860	179.107	13.383
Prophet Day 1 + regressors (Daily)	0.517	9.820	172.597	13.130
Prophet Day 1 alternative config (Daily)	0.003	15.436	356.673	18.885
LSTM Day 1 (Hourly)	0.590	9.096	—	12.838

12 Registry, Automation and Deployment

12.1 Model Registry and Promotion Guard

All three models are registered in the Hopsworks model registry with their metrics, allowing the web interface to display the actual metrics instead of manually writing it. A guard is also being used while retraining the model because during the automation of the training pipeline it is checked whether the new model R2 score beats the old model preventing poor predictions. If the new model performs better than the old model it goes to the registry otherwise the old model stays.

After the registration process, the model is loaded again from the registry and is used to generate the predictions and is compared with the original model predictions making sure they are same. This whole process verifies that the saved model is loading correctly before it can be used in a production.

12.2 Automation

Two GitHub Actions workflows automate the system that is `data_pipeline.py` it executes hourly on schedule collect the new data and upsert in the feature store. Second is the `train_models.py` it executes by day, retrains the model, evaluates the performance of model and if it is working well than the old model so it goes to model registry. The Hopsworks API key is also stored in the GitHub repository secret.

12.3 REST API

The FastApi service loads the registered models and serves seven end points which is deployed on Render.

Table 12: REST API endpoints

Endpoint	Method	Purpose
/	GET	It displays the metadata information and available end points.
/health	GET	Check whether the models are working or not.
/predict	GET	Returns current AQI and day one, two and three forecasts.
/latest-temperature	GET	Returns the latest temperature.
/history	GET	Returns historical AQI for a requested days with predictions.
/predict-file	POST	Accepts a csv file, returns the predicted results (batch backtest).
/metrics	GET	Returns the models metrics for all three with the baseline gains.

12.4 Web Application

Table 13: Pages of the Streamlit dashboard

Page	Contents
Home	Displays current AQI along with temperature and horizon forecasts, alerts and trend chart.
Dashboard	AQI categorical information, historical trend chart, EDA visualizations.
Model Insights	Display metrics for each horizon and SHAP analysis.
Analyze	Here users can upload a csv of dates and receives the predicted values. (batch back testing).

12.5 Application Screenshots

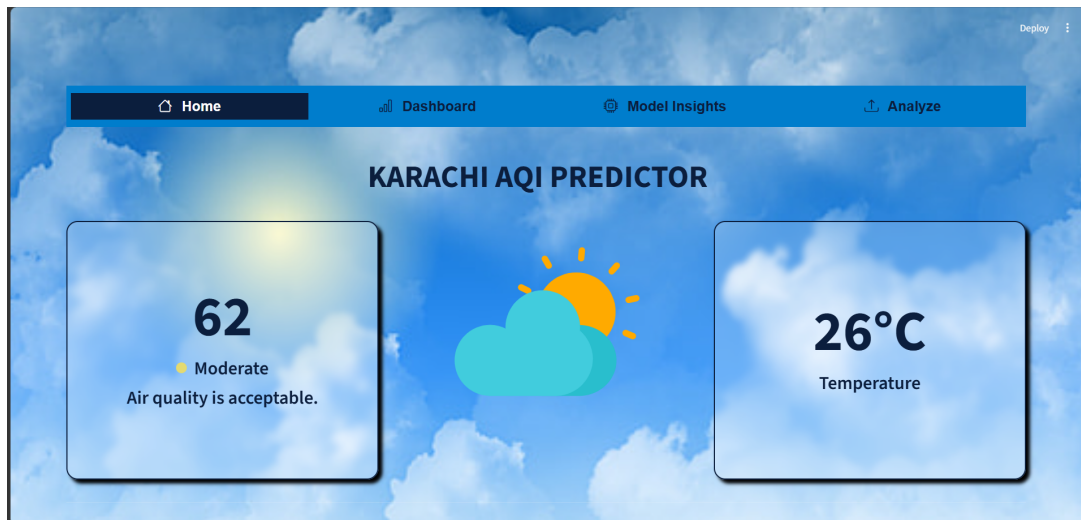


Figure 17: Home page: current AQI with health category, and current temperature.



Figure 18: Home page: one-, two- and three-day forecast cards above the recent trend chart.

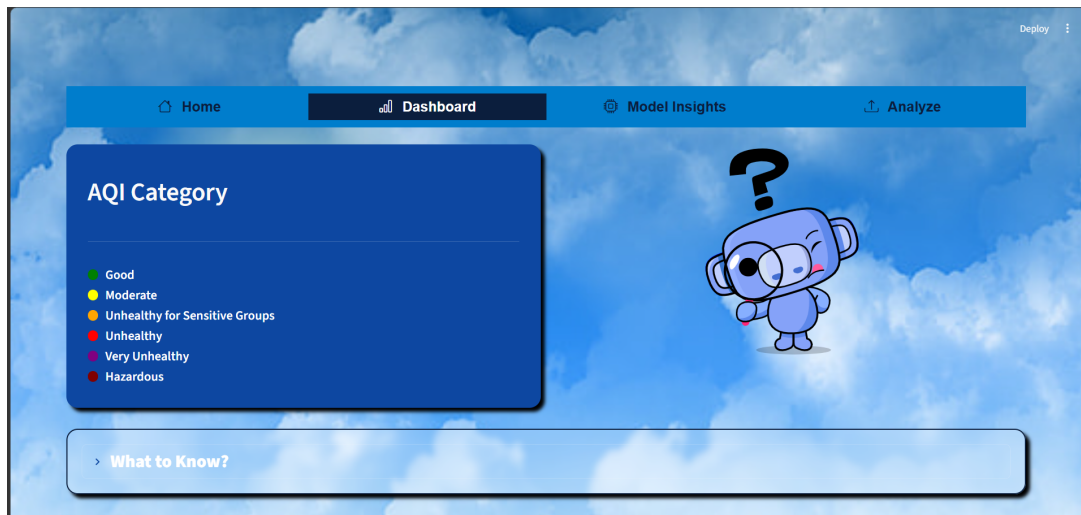


Figure 19: Dashboard: US EPA AQI category legend with colour coding.

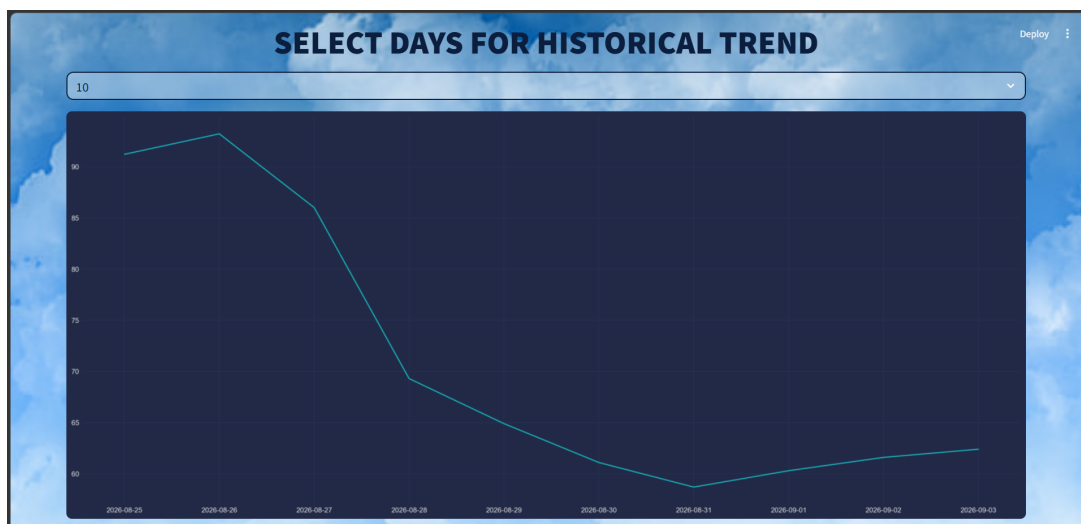


Figure 20: Dashboard: historical AQI trend with a user-selectable window.

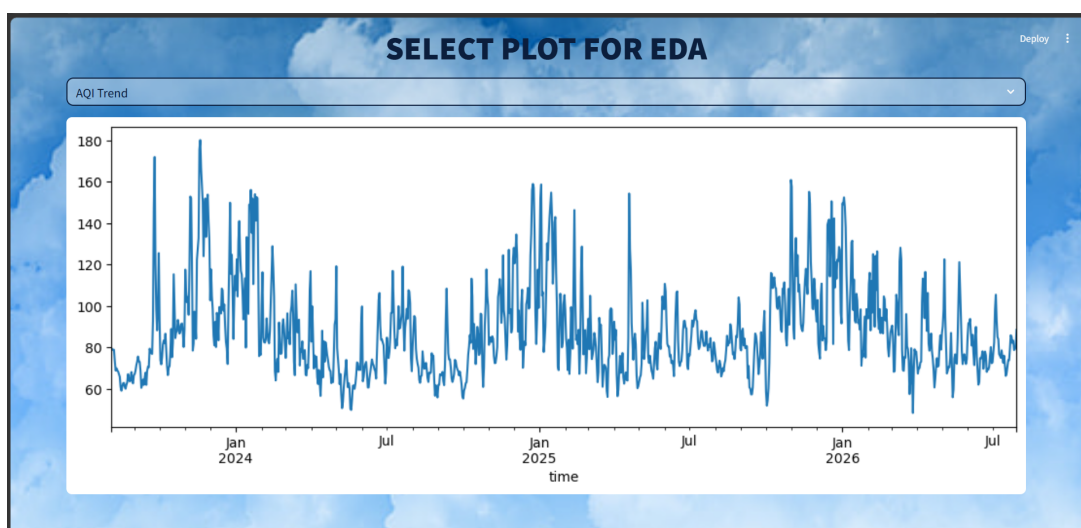


Figure 21: Dashboard: selectable EDA plots, here showing the full three-year AQI series.

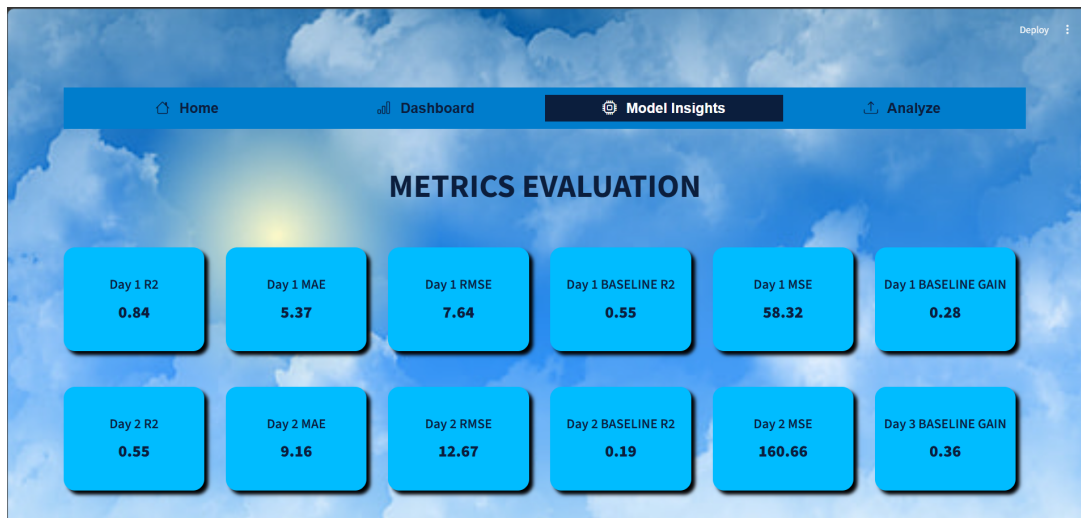


Figure 22: Model Insights: registry metrics for the Day-1 and Day-2 models, including baseline comparison.

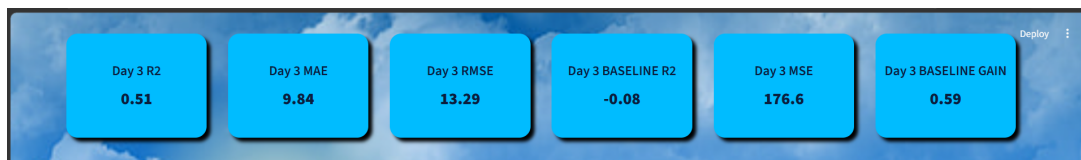


Figure 23: Model Insights: Day-3 metrics. The negative baseline R^2 of -0.08 and the resulting gain of 0.59 are both surfaced.

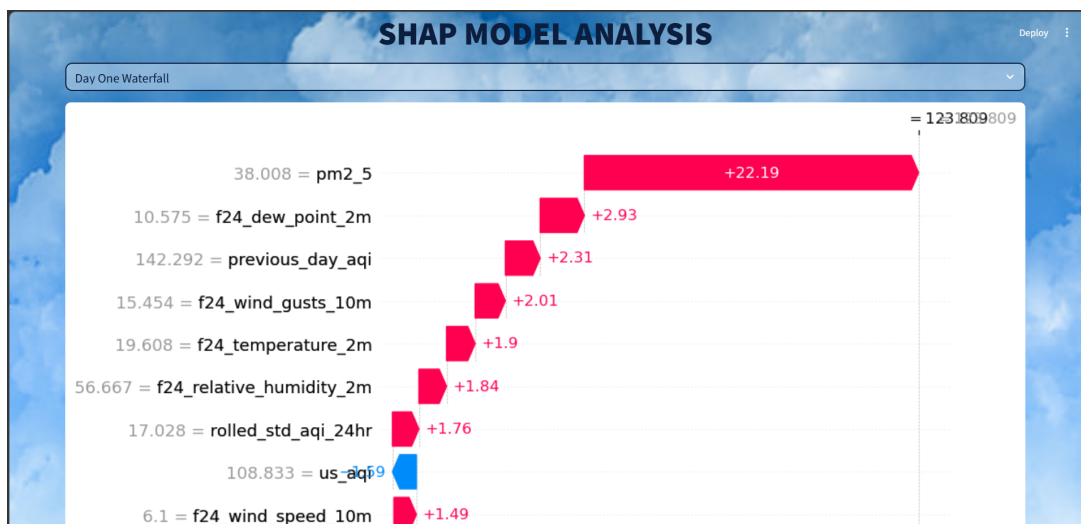


Figure 24: Model Insights: SHAP analysis selector showing a Day-1 waterfall decomposition.

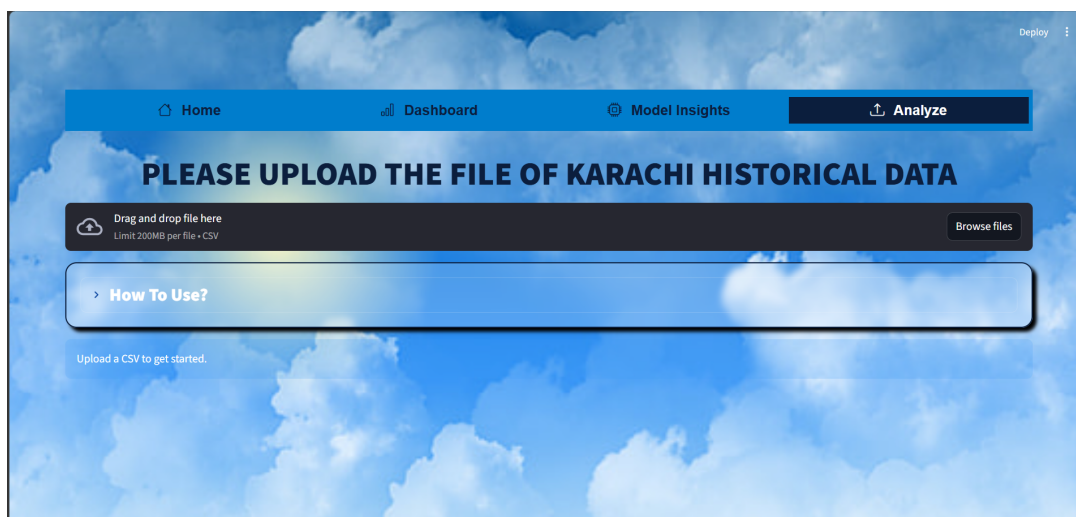


Figure 25: Analyze page: batch backtesting upload interface.

12.6 Interpretability and Alerting

SHAP analysis was conducted for each horizon, includes the beeswarm plots that show the importance and direction of each feature effects, as for waterfall plots that decompose the individual predictions.

On the day one horizon, pm2_5 dominates followed by the current AQI, forecast wind and humidity. As you can see the color gradient shows the direction of effects high values like pm2_5 pushes the prediction upwards and wind speed pushes it downwards, showing that the wind disperses the particulate matters.

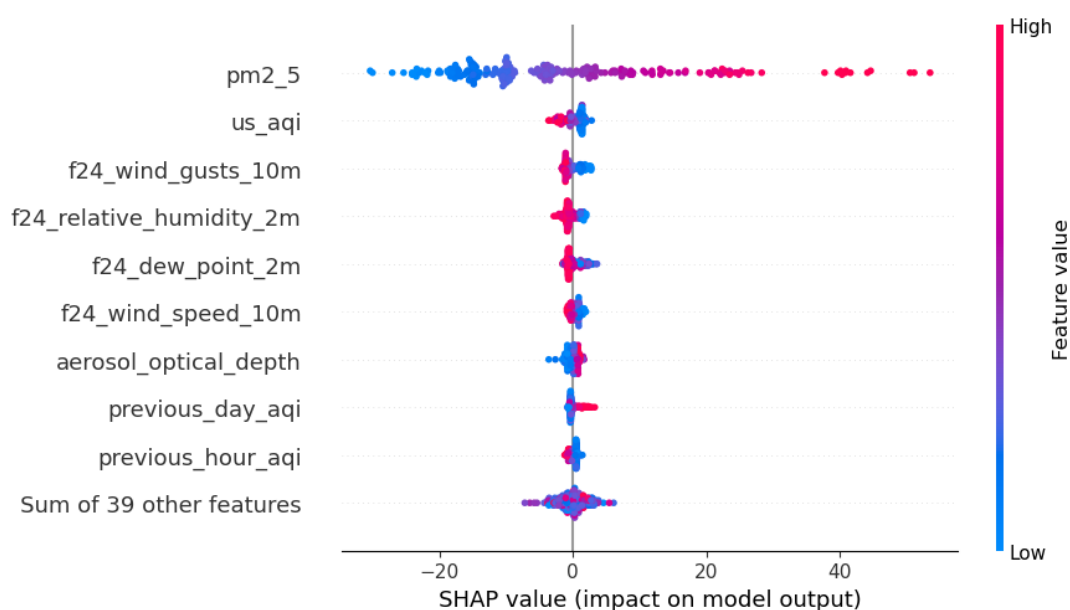


Figure 26: SHAP beeswarm for the Day-1 model. pm2_5 is the dominant driver.

On day two and three, the current pollutant weakens as forecasts grow. Instead, the future weather and seasonal features become more dominated.

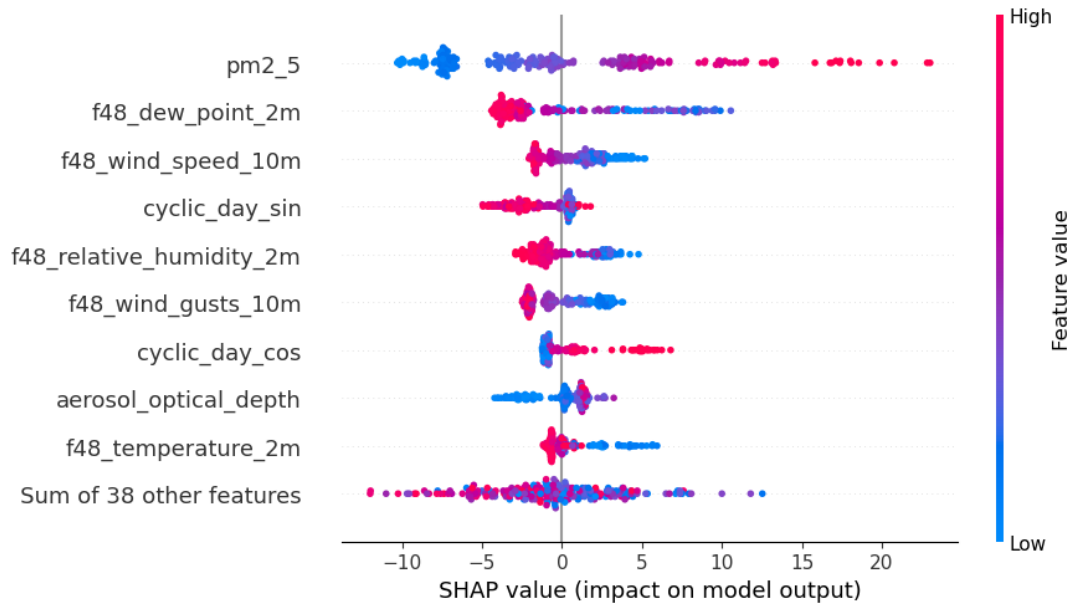


Figure 27: SHAP beeswarm for the Day-2 model.

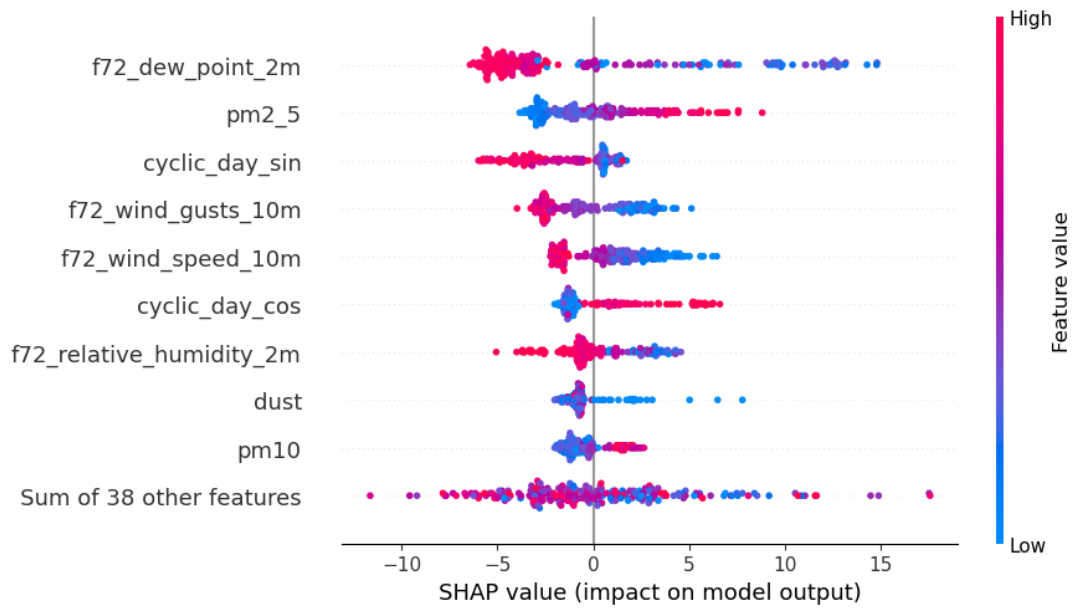


Figure 28: SHAP beeswarm for the Day-3 model.

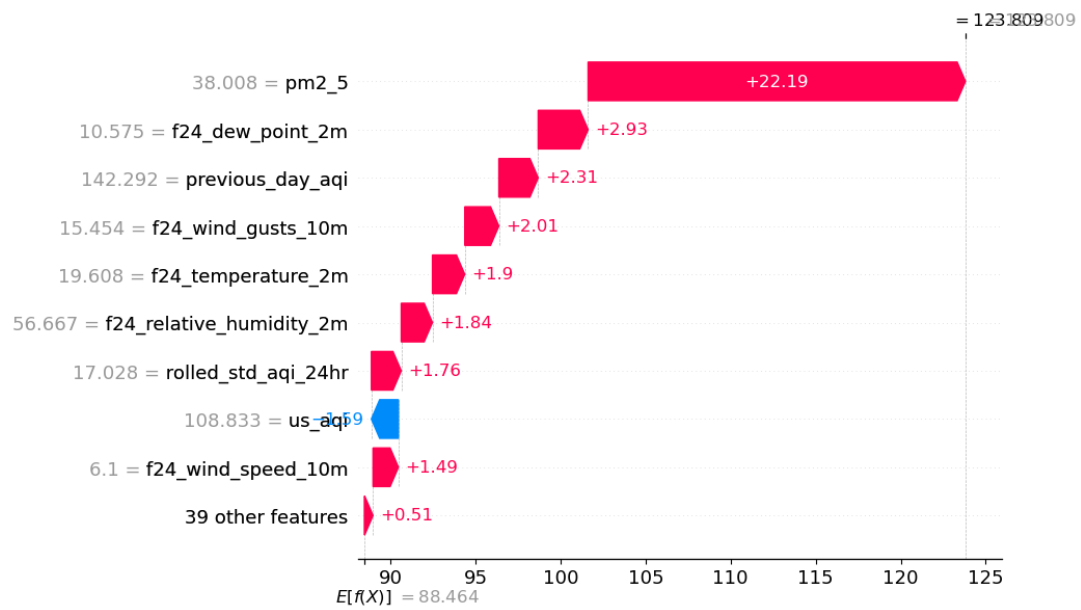


Figure 29: SHAP waterfall for an individual Day-1 prediction.

13 Problems Encountered and Solutions

These sections cover all the problems that I encountered and how I handled it.

13.1 Data Collection and Cleaning

1. Statistical outlier capping destroyed genuine pollution events

Problem: Interquartile-based outlier handling was used as per standard practice. This reduced the maximum ranges AQI from 185 to 139.5, erasing the real time events like smogs episodes. A model trained on such data could never predict hazardous events.

Solution: So, IQR technique is replaced by domain clipping knowledge where pollutants concentrations are clipped at zero to avoid sensor errors, also weather features are clipped based on the physical limitations of Karachi's climate.

2. Row deletion would have corrupted the time series

Problem: The traditional cleaning method like dropping the rows which are out of range, can be bad because lag, rolling and shifted features heavily relies on hourly sequences removing the rows can increase computation errors.

Solution: Clipping replaced deletion entirely. All 26,328 rows are preserved, and hourly continuity is guaranteed, so lag and shift operations remain valid throughout.

3. A six-month gap in boundary_layer_height inside the training region

Problem: boundary_layer_height completely missing from January to July 2024 more than 4000 rows. Data imputation like forward filling would have created unrealistic values corrupting the learning of models.

Solution: So, I must drop the column along with f24_boundary_layer_height and wind_pollution_dispersion, as they were by boundary_layer_height having the same gap issues.

13.2 Feature Engineering and Leakage

4. Cross-horizon target leakage

Problem: The day two and three horizon datasets contain other future days target columns. As an example, day 3 model can see the future day one and two AQI values which should not be available.

Solution: So, in each horizon's dataset I drop the other future horizons target columns to avoid any misleading results.

5. Look-ahead bias in rolling features

Problem: Using the current AQI value the rolling means and standard deviation are calculated. But the current AQI is closely related to target allowing models to see the answer.

Solution: So, all the rolling means and standard deviation features calculated by shifting the AQI by one period. Therefore, the information is used from the past.

6. Shuffled train-test split on time-series data

Problem: The default `train_test_splits` has random shuffling because of this the future data would come in the training data, and past data would come in the testing because of this model would achieve high score, but in real world it does not happen like that.

Solution: The models used `train_test_split` on eighty twenty split techniques with `shuffle` set to false. Eighty percent is the training data and the rest is testing. The chronological split is used in the model.

13.3 Model Performance

7. Hourly models plateaued far below the required threshold

Problem: The hourly based model day 1 XGBoost scored only R2 score 0.625 only, below 0.70 requirement. Even after hyperparameter tuning it did not gain too much.

Solution: So, I moved to the daily aggregation technique where all the data is resampled to daily means instead of hourly. This increases the R2 score to 0.836 by far and removes the short-term noise.

8. Single models could not be improved further at longer horizons

Problem: The single model did not show much improvement in day two and three horizons

Solution: The blend of five models was used, averaging the XGBoost, Cat Boost, Random Forest, Ridge and a bagged SVR to generate the final predictions results because each model make different errors combining their predictions can improve the overall performance.

13.4 Feature Store and Infrastructure

9. Hopsworks project creation failed silently

Problem: The project name problem like projects named `aqi_predictor` and `aqipredictor` did not create project without clear error message. These names were already in use, as Hopsworks requires unique project names.

Solution: I used unique name like, `aqi_karachi_samu_2026` allowing me to create project in Hopsworks.

10. Serverless login failed from a local notebook

Problem: The login connection was failing by calling `hopsworks.login()` which did not work properly in local environment.

Solution: So, I passed explicitly Hopsworks host, port, project name, and API key, the same configuration used in other files too.

11. Free-tier materialization jobs report false failures

Problem: The inserts in feature groups failed because in free tier job status shows failed status even though the data was inserted successfully. Because of this pipeline throws error.

Solution: So, by using `wait=False` insert the pipeline does not block the job status. Also, three attempts of retry are used with the twenty second delay to handle genuine failures instead of a false failure message.

13.5 Deployment

12. Missing Arrow support broke feature-store reads on CI

Problem: As the basic Hopsworks package was installed in my server it gives me runtime failures while reading feature groups on the CI server as it requires `pyarrow` to be installed.

Solution: `hopsworks[python]` was mentioned specifically in the requirements files so this automatically installs the `pyarrow`.

13. The host defaulted to an unsupported Python version

Problem: Render free version tier is set to Python 3.14 by default, for which Hopsworks packages were not compatible, so it causes errors in the Render logs.

Solution I did configuration in my Render environment and set Python to 3.11.9 version so no error occurs.

14. Free-tier idle suspension caused long cold starts

Problem Render free tier biggest issue is that its server can go to sleep after inactivity. After sending the first request must wait until it starts again and download the model files causing request timeouts.

Solution: I used Uptime Robot that monitors and pings the server after every five minutes keeping the server warm and having no timeouts issues.

13.6 Application Layer

15. Comparing a DataFrame against a timestamp

Problem: The system was comparing the entire timestamp of the dataframe instead of between the index and its timestamp raising a Type Error and the API returned 500 statuses with not many details.

Solution: It was corrected using the index of dataframe for date comparison.

16. Forecast rows leaked into historical backtesting

Problem: As seven days before data was collected. That's why feature store contains the rows of future dates because of this the backtesting endpoint returns these dates that have not occurred yet with empty actual values.

Solution: The backtesting endpoint has some logic which filters the dates like removing the future and today date and checking the actual target value availability.

14 Conclusion

Karachi AQI Prediction project delivered is a complete automated machine learning system rather than just a simple model. This end-to-end system collects data from API endpoints, after cleaning and feature engineering is stored in the Hopsworks Feature Store, which gets upsert hourly by the data pipeline. A training pipeline retrains the model daily and registers the new version in the model registry only if it beats the old version. The final predictions are given by the Fast API end points. Finally, all the results are shared on the interactive dashboard along with SHAP analysis and EDA.

The day one model R2 score 0.837, day two and day three are 0.553 and 0.511, both day 2 and day 3 also showed strong improvement as compared to persistence baseline.

The most interesting thing about this project is that the need to identify those errors that could lead to misleading predictions.

15 References

1. Open-Meteo Weather API Documentation: <https://open-meteo.com/en/docs>
2. Open-Meteo AQI API Documentation: <https://open-meteo.com/en/docs/air-quality-api>
3. Hopsworks Complete Documentation: <https://docs.hopsworks.ai/>
4. Fast Api Complete Documentation: <https://fastapi.tiangolo.com/>
5. Streamlet Complete Documentation: <https://docs.streamlit.io/>
6. Render Complete Documentation: <https://render.com/docs>
7. GitHub Actions Complete Documentation: <https://docs.github.com/en/actions>